Machine Learning techniques for malicious PDF detection

Preprint, compiled January 6, 2023

Lorenzo Ippolito, Mattia Rosso, Martino Picasso

Abstract

This project presents a thorough analysis of machine learning techniques for detecting malicious PDF files. By comparing the performance of three binary classification models - K-Nearest Neighbors (KNN), Support Vector Machines (SVM), and Decision Trees - we demonstrate the effectiveness of these approaches in achieving high accuracy and low false negative rates. What makes this project stand out is its focus on the machine learning models themselves, including the careful selection of hyperparameters to optimize their performance. Additionally, the inclusion of KNN as a valid model is rare in the field, and the analysis of evasion techniques adds another layer of depth to the study. Overall, this paper offers a comprehensive look at the use of machine learning for malicious PDF detection and the potential vulnerabilities of these models.

1 Introduction

PDF (Portable Document Format) is a widely used file format for exchanging and sharing documents. It is a versatile format that can contain a variety of different types of content, including text, images, multimedia, and executable code. In this section, we will examine the structure of PDF files and the various elements that can be used to specify the contents of a PDF. One important aspect of PDF structure is the use of keywords, also known as actions or triggers. These keywords are used to specify different types of content, such as JavaScript code, images, and multimedia. In our analysis of malicious PDF files, we have focused on a set of keywords that are commonly known to be discriminant for static PDF malware analysis. By examining the presence of these keywords in a PDF, we can characterize the file and determine whether it may be malicious. In addition to keywords, PDF files can also contain other types of elements, such as fonts, annotations, and form fields. These elements can be used to specify the appearance and interactive features of a PDF, but they can also be exploited for malicious purposes. For example, a PDF may contain a hidden form field that is used to exfiltrate sensitive information from the victim's computer. In summary, the structure of a PDF file is an important factor to consider when detecting malicious content. By analyzing the keywords and other elements present in a PDF, we can gain insight into its contents and determine whether it may be malicious.

History of PDF attacks The flexibility and widespread adoption of PDF also make it a popular choice for delivering malicious content. In this chapter, we will review the history of PDF attacks and the various types of malicious content that have been delivered via PDF files. The first known PDF attack occurred in 2005, when a malicious PDF file was used to deliver the Trojan horse malware known as "Zotob." This malware exploited a vulnerability in Microsoft Windows and was used to take control of infected computers. Since then, PDF files have been used to deliver a wide range of malware, including ransomware, spyware, and adware. In addition to malware, PDF files have also been used to deliver phishing attacks and other forms of cyber threats. For example, a PDF file may contain a link to a fake login page or a form that asks the user to enter sensitive information, such as passwords or credit card numbers. By tricking the user into entering this information, the attacker can gain access to their accounts or steal their identity. Another common type of PDF attack is the use of "exploit kits." These kits are collections of tools that can be used to exploit vulnerabilities in software and deliver malware via PDF files. One well-known exploit kit is "Blackhole," which was used to deliver a variety of malware, including the "Citadel" banking Trojan.

2 Related work

See Section 2.

2.1 Related work: second level

2.1.1 Related work: third level

Paragraph [1, 2] [3].

Hasselmo, et al. (1995) investigated...

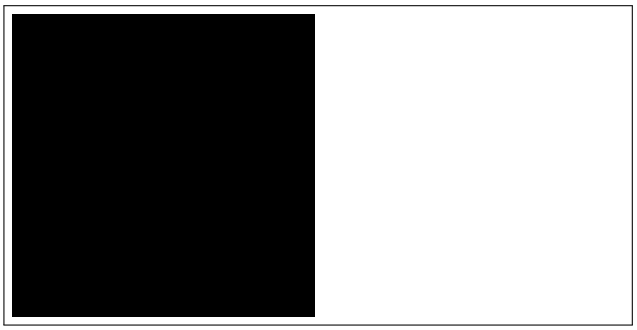

Figure 1: Sample figure caption.

- a
- b
- c

3 Datasets and features

In this chapter, we will discuss the data sources and extraction process for our machine learning model for detecting malicious PDFs. As the first step in building our model, we needed to obtain a dataset of PDFs to use for training and evaluation. After reviewing several options, we decided to use the Contagio dataset for our work. The Contagio dataset is a collection of

*correspondence: email@institution.edu

Publication doi

Preprint doi

malicious PDF files that has been widely used for research and testing purposes. The dataset was created by the security firm Mandiant and contains about 20000 PDF samples, including malware, phishing attacks, and other types of malicious content. The samples in the dataset are organized into various categories, such as exploit kits, banking Trojans, and ransomware, and are representative of the types of threats that are commonly encountered in the wild. This dataset has been cited in many publications on the topic of malicious PDF detection, and it provides a balanced number of both malicious and benign PDFs. By using this dataset, we can compare our results to those reported in previous publications and see if our models are performing better or worse. Next, we needed to extract the data from the PDFs in order to use it for training and evaluation. We chose to perform a static analysis, which does not require opening the PDFs in a reader. Instead, we used a tool called PDFId to extract relevant information from the PDFs. PDFId characterizes each PDF by extracting the number of occurrences of specific keywords. This allows us to map each PDF into an array of 22 elements, with each element representing the count of a specific keyword. We are therefore working with samples of 22 integer features. We report here the features used with a short summary of their meaning and use:

- obj: It is used to specify the start of an object in a PDF file. An object can be a variety of types of data, such as text, images, or executable code.
- endobj: It is used to specify the end of an object in a PDF file. It marks the end of the data associated with a particular object.
- stream: It is used to specify the start of a stream of data in a PDF file. A stream is a contiguous sequence of data that is usually compressed and can be a variety of types, such as text, images, or audio.
- endstream: It is used to specify the end of a stream of data in a PDF file. It marks the end of the data associated with a particular stream.
- xref: It is used to specify a cross-reference table in a PDF file. The cross-reference table contains information about the objects in the PDF file, including their location and size.
- trailer: It is used to specify the trailer of a PDF file. The trailer contains information about the structure of the PDF file, such as the location of the cross-reference table and the root object.
- startxref: It is used to specify the starting position of the cross-reference table in a PDF file. It is typically located at the end of the file and is used to locate the objects in the PDF file.
- /Page: It is used to specify a page object in a PDF file. A page object represents a single page in the PDF document and contains information about the layout and content of the page.
- /Encrypt: It is used to specify that a PDF file is encrypted. Encrypted PDF files are protected with a password and cannot be viewed or edited without the correct password.
- /ObjStm: It is used to specify an object stream in a PDF file. An object stream is a stream of data that contains multiple objects, which are compressed and stored in a single stream to save space.
- /JS: It is used to specify that a PDF file contains JavaScript code. JavaScript is a programming language that can be used to create interactive features in a PDF file, but it can also be used for malicious purposes.
- /JavaScript: It is similar to the /JS keyword and is used to specify that a PDF file contains JavaScript code.
- /AA: It is used to specify that a PDF file contains additional actions. Additional actions are additional instructions or triggers that are executed when certain events occur, such as when the PDF file is opened or a form is submitted.
- /OpenAction: It is used to specify an action that is executed when a PDF file is opened. This action can be used to execute code or perform other tasks.
- /AcroForm: It is used to specify that a PDF file contains interactive form fields. Form fields can be used to create interactive forms that can be filled out by the user, but they can also be used for malicious purposes, such as exfiltrating sensitive information from the user.
- /JBIG2Decode: It is used to specify that a PDF file contains JBIG2 encoded data. JBIG2 is a compression algorithm that is used to compress black and white images, and is often used in PDF files to reduce the size of scanned documents.
- /RichMedia: It is used to specify that a PDF file contains rich media content, such as audio or video. Rich media can be used to enhance the interactive experience of a PDF file, but it can also be used to deliver malware or other malicious content.
- /Launch: It is used to specify an action that is executed when a PDF file is opened. This action can be used to launch a program or open a file, and is often used to deliver malware or other malicious content.
- /EmbeddedFile: It is used to specify that a PDF file contains an embedded file. An embedded file is a file that is stored within the PDF file and can be extracted and accessed by the user.
- /XFA: It is used to specify that a PDF file contains an XFA (XML Forms Architecture) form. XFA forms are used to create interactive forms that can be filled out by the user, and are often used in conjunction with AcroForms.
- /Colors: It is used to specify that a PDF file contains more than 16 million colors. This is typically an indication of high-quality images or graphics, but it can also be used to identify malicious PDF files that use large images or graphics to hide their contents.

The output of PDFId is then converted into a CSV file, which can be easily accessed using the numpy and pandas libraries. This CSV file serves as the basis for our data extraction pipeline, which we have implemented using the functions provided by PDFId. It is important to note that PDFId is a tool specifically designed for the analysis of malicious PDFs, as the 22 keywords it extracts are well known to be discriminant for PDF malware

analysis. These keywords do not represent the full set of possible keywords that may appear in a PDF file. For our classification task, we will be performing a binary classification, with malicious PDFs being labelled as positive samples and benign PDFs being labelled as negative samples. This allows us to easily train and evaluate our models, as we only need to predict whether a given PDF is malicious or not.

3.0.1 Model validation

In this chapter, we will discuss the methods we used to validate the models developed in our study of machine learning techniques for malicious PDF detection. Model validation is an important step in the development of any machine learning model, as it allows us to assess the performance of the model and identify any issues or biases that may affect its accuracy. One method we used for model validation was a static train-test split. In this approach, we split all of the samples in our dataset into two sets: a training set and a test set. The training set, which consists of 80% of the samples, is used to train and validate the models using a k-fold cross validation approach. The test set, which consists of 20% of the samples, is not used to make decisions during the model building phase, but is instead used to make final considerations about the performance of the models trained with the best parameters/hyperparameters. To evaluate the performance of our models, we used two measures: the False Negative Rate (FNR) and Accuracy. The FNR is defined as

$$FNR = \frac{FN}{FN+TP}$$

We aimed to minimize the FNR as much as possible, as False Negatives can be particularly damaging in the context of malicious PDF detection. False Negatives can lead to the failure to detect and prevent attacks and can have serious consequences for individuals and organizations. At the same time, we also took into consideration the value of Accuracy, as the penalization of False Negative errors should not result in an excessively high number of False Positive errors. It is important to find a balance between minimizing False Negatives and minimizing False Positives, as both can have negative consequences. False Positives can lead to unnecessary alerts and can waste resources and can also erode trust in the system. In addition to the static train-test split and the measures of FNR and Accuracy, we also used other techniques to validate our models. For example, we used cross-validation to ensure that our models were not overfitting to the training data and to identify the optimal hyperparameters for each model.

4 SUPPORT VECTOR MACHINES

Support Vector Machines are linear classifiers that look for maximum margin separation hyperplanes. The primal formulation of the soft-margin SVM problem consists in minimizing the function:

$$\begin{gathered} J(w, w_0) = \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i \\ \text{s.t. } z_i(w^T x_i + w_0) \geq 1 - \xi_i \\ \text{and } \xi_i \geq 0 \end{gathered}$$

To define the hard margin SVM problem the penalty term $C\sum_{i=1}^{n}\xi_i$ has been added to the soft margin SVM problem. ξ_i are slack variables and estimate the number of missclassified samples. The hyperparameter C is meant to define the strongness of this penalization. We also introduce the dual problem that, besides the fact that is the one that *sklearn* optimizes, is also useful to explain how the adversarial samples will be crafted in section 8.

$$\begin{gathered} J_D(\alpha) = \sum_{i=1}^{n}\alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j z_i z_j x_i^T x_j \\ \text{s.t. } 0 \leq \alpha_i \leq C \\ \text{and } \sum_{i=1}^{n}\alpha_i z_i = 0 \end{gathered}$$

The function J_D has to be maximized and a set of n Lagrangian multipliers α has been introduced to embed the primal constraints.

4.1 Linear SVM

We implemented a linear SVM classifier using the *sklearn* library. The samples x_i are features vectors of 21 elements (the one extracted from PDFiD).

4.1.1 Experiments

We have cross-validated the linear model by trying 10 different values for the C hyperparameter in the range $[10^{-6}, 10]$. As explained in section 3 we optimized for the False Negative Rate (FNR) with the constraint of obtaining a high accuracy anyway.

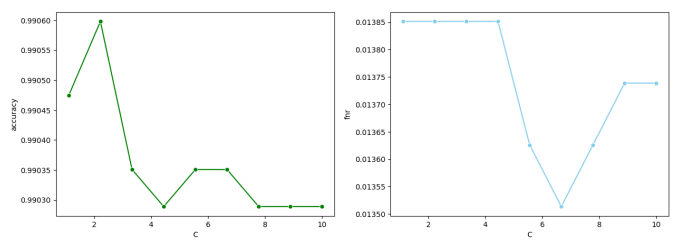

Figure 2: The plots show how the accuracy (left) and the FNR (right) change when the value of C hyperparameter changes

The plot in figure 2 show that the variations in terms of Accuracy and FNR when different values of C are used to train the linear SVM are not that evident. Here the results obtained with $C = 10^{-6}$ where not reported because of the significatively worse results (88.67% of accuracy and 2.94% of FNR).
Summarizing the best results we obtained:

C	Accuracy	FNR
2.22	99.06%	1.38%
6.67	99.03%	1.35%

Optimizing for the FNR the hyperparameter we have selected is $C = 6.67$.

4.2 Kernel SVM

What it makes the SVMs particularly powerful is the way how the dual problem is defined. We are indeed able to embed a non-linear decision function that computes the scalar product between x_i and x_j in a higher dimensional space without needing

to explicitly compute the features expansion. A kernel function is defined as

$$k(x, y) = \Phi(x)^T \Phi(y)$$

And the J_D function is modified as:

$$J_D(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j z_i z_j k(x_i, x_j)$$

Also the inference step is modified accordingly:

$$y_t = w^T x_t + w_0 = \sum_{i=1}^{n} \alpha_i z_i k(x_t, x_i)$$

We have implemented a Radial Basis Function (RBF) kernel that is defined as:

$$k(x, y) = e^{-\gamma \|x-y\|^2}$$

4.2.1 *Experiments*

The cross-validation of the model has now to take into account two different hyperparameters (γ for the RBF kernel and C for the dual SVM problem). This time we exploited a grid search k-fold cross validation that makes use of the cross validation to train the model with each possible combinations of hyperparameters. The hyperparameters values we tried are:

- γ: 10 values in $[10^{-6}, 1]$
- C: 10 values in $[10^{-6}, 10]$

We now investigate the influence of both hyperparameters. What it's not displayed and analyzed in the plots is what happens with $C = 10^{-6}$ and $\gamma = 10^{-6}$, parameters under which the classifier performs a very poorly in terms of accuracy (around 54%) despite achieving a FPR equal to 0%. However, this is not better than a dummy classifier that safely detects every PDF as malicious so it's not of interest in this project.

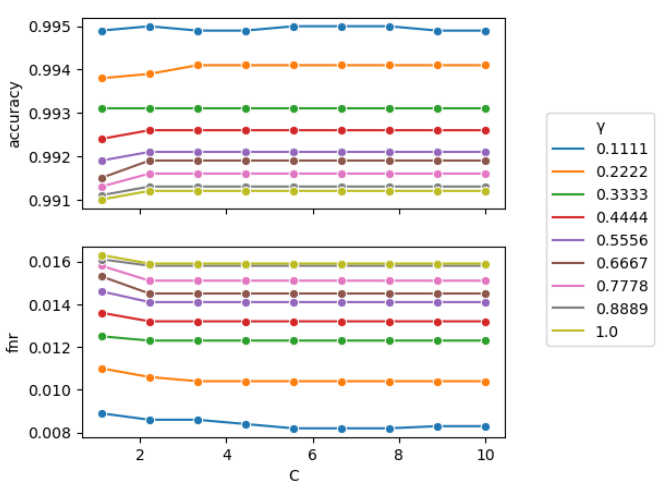

Figure 3: Accuracy (top) and FNR (bottom) when γ (colored lines) changes and C changes

In figure 3 it's clear that smaller values of γ lead to obtain better results both in terms of Accuracy and FNR. The γ parameter of the RBF kernel controls the width of the kernel, it determines how far the influence of a single training example reaches, and how smooth the decision boundary is. A lower value of gamma means a larger, less concentrated kernel, which can lead to smoother decision boundaries.

We also investigated the role of the C hyperparameter.

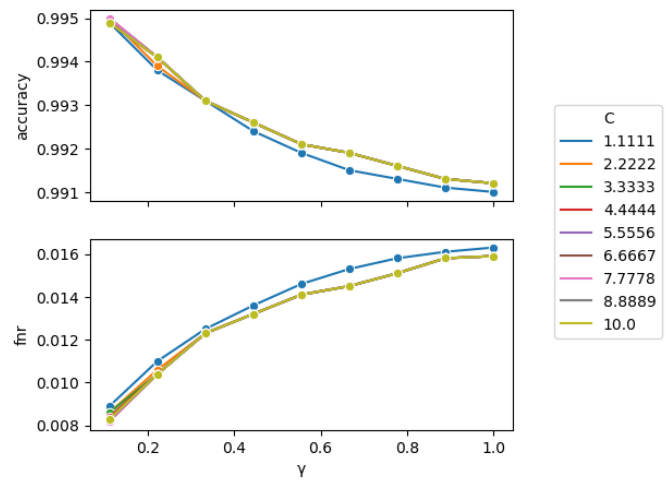

Figure 4: Accuracy (top) and FNR (bottom) when C (colored lines) changes and γ changes

As we can observe in figure 4 the value of the C hyperparameter has less influence on the model than γ. The C hyperparameter controls the trade-off between the simplicity of the model (low bias, high variance) and the training error (low variance, high bias). A higher value of C means a higher penalty for misclassification while a lower value of C means a lower penalty for misclassification.
When the value of γ is sufficiently small the accuracy reaches the best values and the same happens for the FNR. The choice of C has clearly less impact on the performances.

Overall the best values we have found from cross validation are:

C	γ	**Accuracy**	**FNR**
5.56	0.11	99.50%	0.82%

4.3 *Performances on the test set*

Finally, given the set of best hyperparameters found through cross validation on the entire training set we are going to asses the performances achieved on the test set.

4.3.1 *Linear SVM*

C	**Accuracy**	**FNR**
6.67	99.06%	1.58%

Table 1: Linear SVM - Performances on the test set

4.3.2 *Kernel SVM*

Secondly we train a RBF SVM on the entire training set and we do the predictions on the entire testing set.

C	γ	Accuracy	FNR
5.56	0.11	99.50%	0.82%

Table 2: RBF SVM - Performances on the test set

5 DECISION TREE AND RANDOM FOREST

The decision tree is a popular classification technique in which predictions are made in a sequence of single-attribute tests. A decision tree consists of three types of nodes:

- decision nodes (squares)
- chance nodes (circles)
- end nodes (triangles)

Decision trees are used to evaluate the expected values or expected utility of competing alternatives. Each internal node in a decision tree represents a test on an attribute, and each branch represents the outcome of the test. The paths from the root to the leaf nodes represent classification rules, and the leaf nodes represent class labels. In this test, we evaluated the performance of a decision tree using various parameters such as tree depth, minimum number of samples per leaf, and maximum number of features. We found that adjusting these parameters had a significant impact on the false negative rate and the accuracy of the model. For example, as shown in Figure 1, we found that the optimal tree depth parameter was 15, which provided the best balance between model performance and overfitting.

5.1 Random forest

Random Forest is a machine learning algorithm that is used for both classification and regression tasks. It is an ensemble method, which means that it combines the predictions of multiple decision tree models to make a more accurate and stable prediction. To train a Random Forest model, multiple decision trees are trained on different samples of the data and their predictions are combined. This helps to reduce overfitting and improve the model's generalization ability. The algorithm is also robust to missing values, making it a versatile and widely used machine learning tool. In our analysis we chose this model because of its high accuracy and stability. It is known for performing well on a wide range of datasets and tasks. Also with Random Forest, we evaluated the performance varying parameters and in Figure we can see the results for maximum trees depth.

6 KNN

Algorithm

6.1 Experiments

7 FINAL RESULTS

8 ATTACKING THE CLASSIFIERS

9 CONCLUSIONS AND FUTURE WORKS

10 CONTRIBUTIONS

REFERENCES

[1] George Kour and Raid Saabne. Real-time segmentation of on-line handwritten arabic script. In *Frontiers in Handwriting Recognition (ICFHR), 2014 14th International Conference on*, pages 417–422. IEEE, 2014.

[2] George Kour and Raid Saabne. Fast classification of handwritten on-line arabic characters. In *Soft Computing and Pattern Recognition (SoCPaR), 2014 6th International Conference of*, pages 312–318. IEEE, 2014.

[3] Guy Hadash, Einat Kermany, Boaz Carmeli, Ofer Lavi, George Kour, and Alon Jacovi. Estimate and replace: A novel approach to integrating deep neural networks with existing applications. *arXiv preprint arXiv:1804.09028*, 2018.